Algorithms: Design and Analysis, Part I

Mihir Rao

Contents

1 Introduction and Merge Sort **2**
1.1 Algorithms and analysis 2
1.2 Integer multiplication 2
1.3 Merge sort 3

Lecture 1: Introduction and Merge Sort

1.1 Algorithms and analysis

An algorithm specifies a procedure for solving a problem on every valid input. A useful analysis asks both whether it returns the required answer and how its resource use grows with input size. The choice of algorithm can change what sizes of problem are practical, even on the same machine.

For general-purpose routines, worst-case analysis bounds the work on *every* input of a given size. This is different from a benchmark on selected inputs or an average over a specified distribution. Asymptotic analysis emphasizes growth for large inputs, suppressing constant factors and lower-order terms. This makes comparisons less dependent on implementation and hardware, though constants and actual input sizes still matter in practice.

1.2 Integer multiplication

The grade-school method for multiplying two n-digit numbers takes $O(n^2)$ single-digit operations. A recursive decomposition suggests a different approach. For even n, let $B = 10^{n/2}$ and write

$$x = aB + b, \qquad y = cB + d, \qquad xy = acB^2 + (ad + bc)B + bd.$$

Four recursive multiplications compute ac, ad, bc, bd, but this alone does not improve the quadratic growth. Karatsuba multiplication computes only ac, bd, and $(a + b)(c + d)$ recursively, recovering the cross term as $(a + b)(c + d) - ac - bd$. Shifts, additions, and subtractions take linear work per call. Up to rounding and the possible extra carry digit, the recurrence is $T(n) = 3T(n/2) + O(n)$, giving $O(n^{\log_2 3})$ work. The gain comes from reducing the number of recursive multiplications, not merely splitting the input.

1.3 Merge sort

Given an array of n numbers, MERGE SORT recursively sorts each half and merges the two sorted results. Arrays of length at most one are already sorted. To merge, compare the first unconsumed element of each half, emit the smaller, and advance that half's pointer; when one half is exhausted, append the rest of the other. A consistent tie rule handles equal keys.

The merge is correct because the smallest remaining item must be at the front of one of the sorted halves. Induction on the input size then proves that both recursive results are sorted and their merge is a sorted permutation of the input. Every element is emitted once, so merging m elements takes $O(m)$ work.

For n a power of two, the recursion tree has 2^j subproblems of size $n/2^j$ at level j. The total merge work per level is $O(n)$, and there are $\log_2 n$ merge levels. Hence the worst-case running time is $O(n \log n)$, including the linear total work at the leaves. Uneven splits change only the rounding and preserve the same bound.

Source. Tim Roughgarden, Algorithms: Design and Analysis, Part I, Part 1 slides.